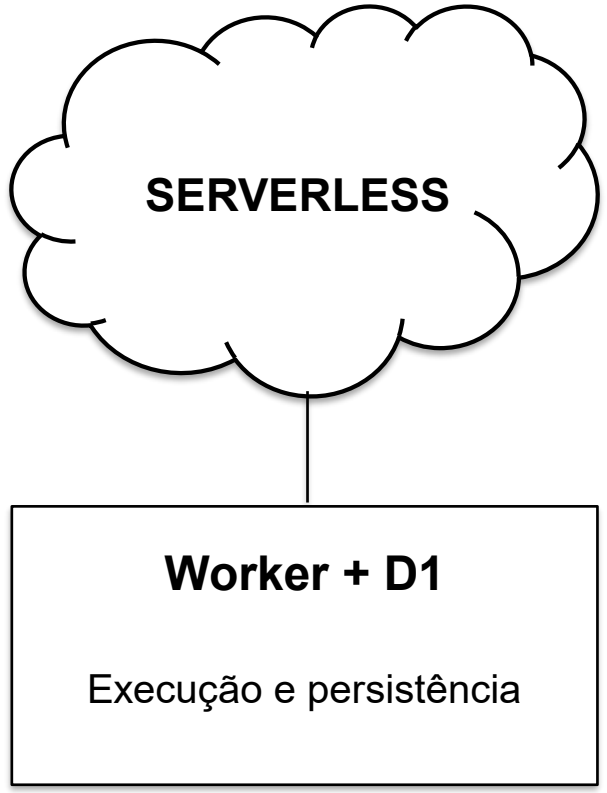
Computação Serverless

CloudExpense

Smart Personal Finance

Uma aplicação de gestão financeira pessoal com Cloudflare Workers e Cloudflare D1.



Ayrton Nhaguaga · Abelardo Miambo · Edmilson Conge

Turma: Lecc31 | Docente: Sergio Sambo

Percurso da apresentação

01 Contexto e objectivos

Introdução, problema e justificativa.

02 Fundamentos

Serverless, FaaS e responsabilidades.

03 Implementação

Arquitectura, dados, interface e segurança.

04 Avaliação

Demonstração, testes e conclusão.

Introdução: do registo à informação

- Registar receitas e despesas permite organizar os movimentos financeiros.
- Categorias e relatórios transformam registos em informação útil.
- Uma aplicação web exige interface, processamento e persistência.

Questão tecnológica

Como executar essa lógica numa infraestrutura gerida?

Caso de estudo

CloudExpense: uma aplicação completa para demonstrar serverless.

Fonte: descrição e objectivos do projecto CloudExpense.

Problema e pergunta de investigação

Necessidade funcional

- Centralizar receitas, despesas e categorias.
- Calcular saldos e consultar o histórico com consistência.

Desafio de implementação

- Executar lógica sob demanda.
- Persistir dados e validar entradas sem um backend Node.js permanente.

Como construir uma aplicação financeira funcional com uma arquitectura serverless?

Justificativa e relevância

01

Aplicação prática

Um domínio claro: receitas, despesas, saldo e categorias.

02

Aprendizagem

Tornar visível o fluxo entre interface, função e persistência.

03

Engenharia

Integrar validação, integridade, testes e documentação.

A contribuição é uma implementação demonstrável, acompanhada de evidências locais.

Objectivo geral e objectivos específicos

Desenvolver uma aplicação web de gestão financeira pessoal baseada em serverless, com execução sob demanda e persistência de dados.

01

Implementar

API REST, regras de negócio e D1.

02

Integrar

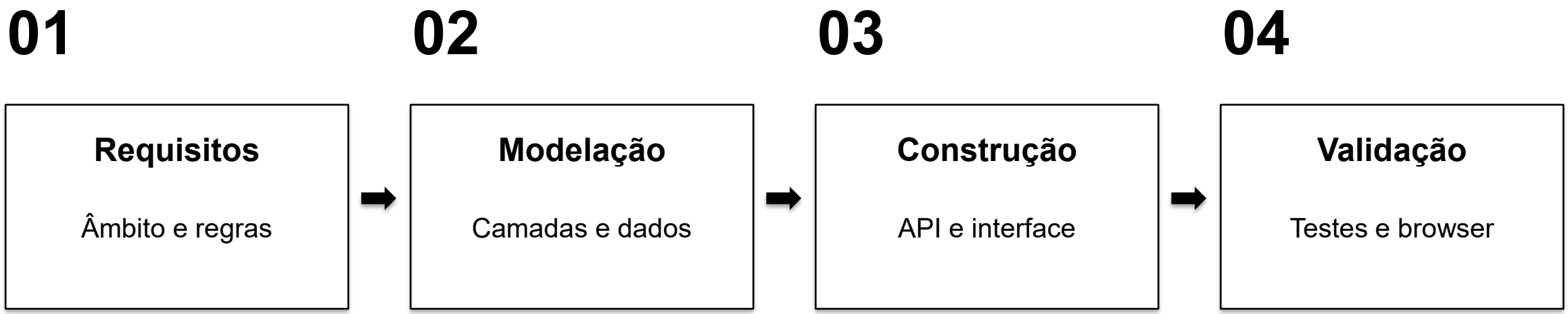
React, CRUD, filtros e gráficos reais.

03

Validar

Testes, logs e documentação de deploy.

Metodologia de desenvolvimento

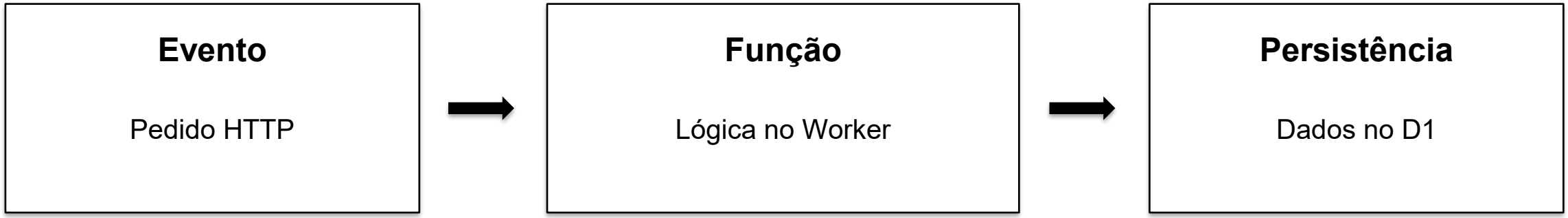


Iteração: implementar → verificar → corrigir → verificar novamente

Artefactos: código, migrações, testes, documentação e apresentação.

O que é Computação Serverless?

Infraestrutura gerida pelo fornecedor; lógica da aplicação activada por eventos.



FaaS: Function as a Service

Serverless não significa ausência física de servidores.

Referências: [1] funcionamento de Workers; [2] visão geral do D1.

Hospedagem tradicional e serverless

Aspecto	Servidor tradicional	CloudExpense serverless
Execução	Processo numa máquina	Handler HTTP no Worker
Infraestrutura	Provisionamento e manutenção	Execução gerida pela Cloudflare
Escala	Instâncias e balanceamento	Gerida, sujeita a limites
Dados	Banco local ou serviço externo	D1 gerido e persistente
Programador	Código + operação do ambiente	Código, dados e segurança

A responsabilidade muda de distribuição; não desaparece.

Referência: [1]. Comparação conceptual; sem benchmark de custos ou desempenho.

Tecnologias e responsabilidades

Interface

React + TypeScript + Vite

Tailwind CSS, Recharts e TanStack Query

Lógica da aplicação

Cloudflare Workers + Hono

Zod e API REST em JSON

Persistência

Cloudflare D1 / SQLite

SQL parametrizado e migrações

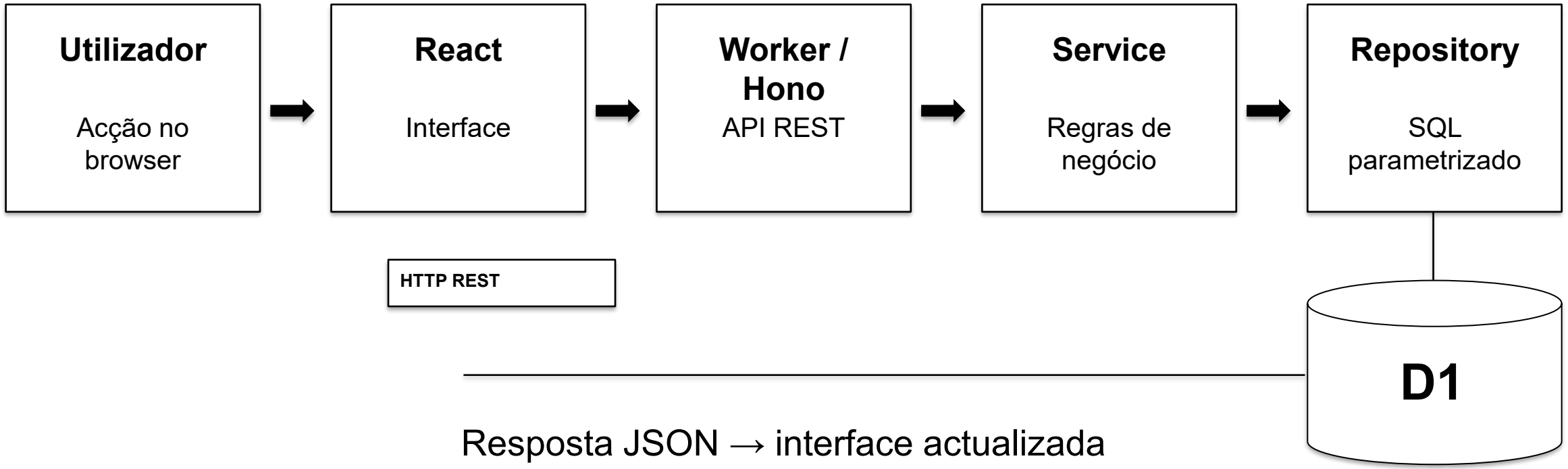
Desenvolvimento e testes

Wrangler, Vitest e Testing Library

Playwright para os fluxos no browser

Node.js executa as ferramentas; o backend de produção executa no Worker.

Arquitectura da aplicação



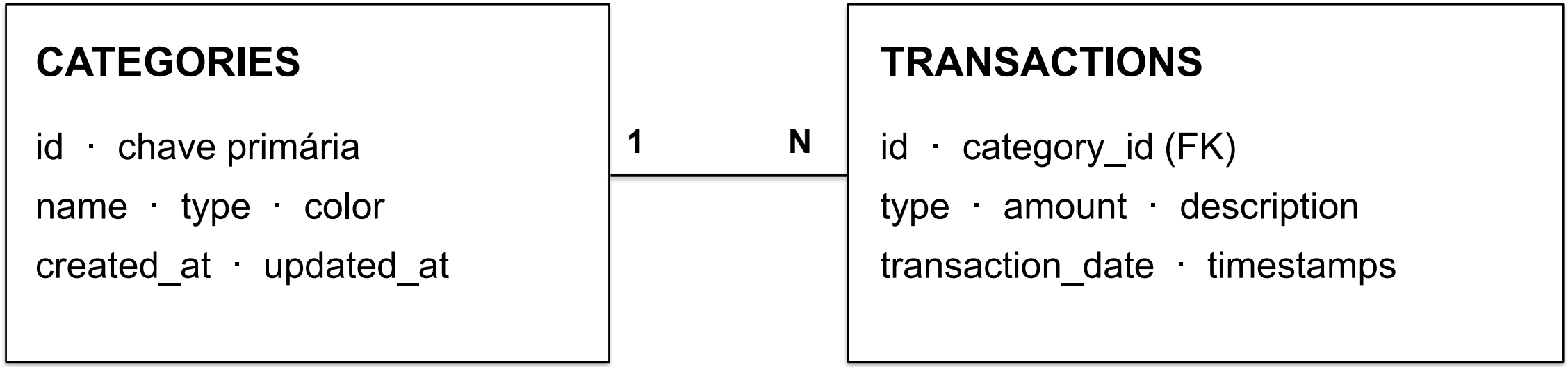
Produção: Cloudflare Access + Workers Static Assets + API no mesmo domínio.

Fonte: docs/ARCHITECTURE.md. Protecção Access e publicação remota ainda pendentes.

O percurso de uma transacção

01	**Acção**	O utilizador preenche e submete o formulário.
02	**Requisição**	React envia POST /api/transactions.
03	**Validação**	Worker verifica dados, montante e categoria.
04	**Persistência**	Repository executa INSERT parametrizado no D1.
05	**Resposta**	HTTP 201; a interface consulta novamente os totais.

Modelo de dados e integridade

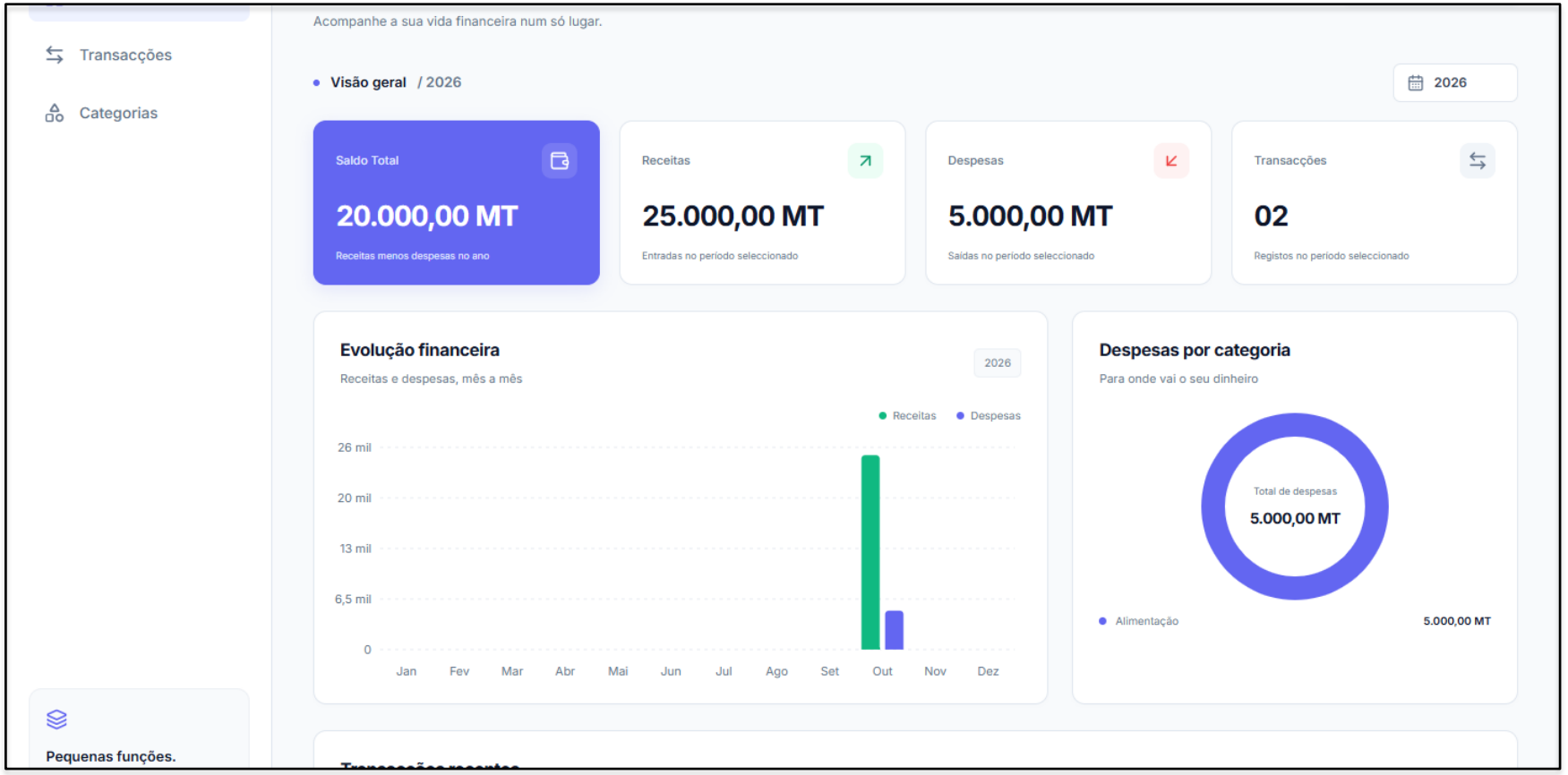


1.500,50 MT → 150050 centavos (INTEGER)

Chaves estrangeiras + CHECK + triggers + consultas parametrizadas.

Fonte: worker/migrations/0001_initial.sql e shared/schemas.ts.

Dashboard: informação financeira visível

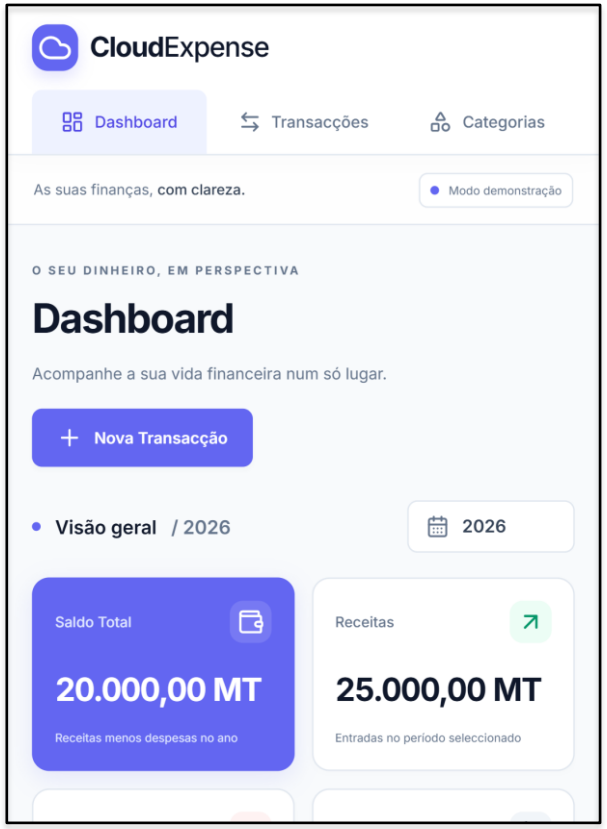


Dados reais da API

- Saldo do ano
- Receitas e despesas
- Distribuição por categoria

Imagem: captura real do CloudExpense em teste local; dados de demonstração, não dados pessoais.

Funcionalidades e experiência de utilização



- Receitas e despesas: criar, editar e eliminar com confirmação.
- Categorias: nome, tipo e cor, preservando relações existentes.
- Pesquisa, filtros combinados e paginação.
- Layout responsivo, validação, loading e mensagens de erro.

Imagem: captura mobile do teste E2E. Fonte funcional: README.md e docs/TESTING.md.

Segurança em várias camadas

01

Entrada

Zod, limites de corpo e rejeição de origens de escrita indevidas.

02

Dados

SQL parametrizado, chaves estrangeiras e regras de integridade.

03

Acesso

JWT Access validado; produção bloqueia se faltar configuração.

MVP académico partilhado: não destinado a dados financeiros privados reais.

Referência: [3] validação de JWT Access. Evidência local: testes do Worker.

Demonstração: do movimento ao saldo

Receita		Despesa		Saldo
25.000 MT	−	5.000 MT	=	20.000 MT

- Recarregar a página: os dados permanecem no D1 local.
- Editar ou eliminar: os totais são recalculados pelo backend.

Fonte: testes locais de integração e E2E. Montantes temporários de demonstração.

Testes e resultados obtidos

23

Frontend

Formulários, moeda, estados e cliente HTTP.

32

Worker + D1

CRUD, integridade, filtros, relatórios e Access.

02

Browser / E2E

Percursos completos em desktop e mobile.

Também aprovados: TypeScript, migração local, build e Wrangler --dry-run.

Limite da evidência: sem teste de carga nem validação de um deploy remoto.

Fonte: docs/TESTING.md — execução local em 11/10/2026.

Como observar a execução serverless

No ambiente local

- Network: pedido HTTP e resposta JSON.
- Health check: Worker consulta o D1.
- Logs: método, rota, status e requestId.

Depois da publicação

- Wrangler tail e Workers Observability.
- Invocações, erros e consumo na conta.
- Verificar Access com utilizadores autorizados.

Tempo HTTP no cliente ≠ tempo de CPU do Worker.

Vantagens, custos e limitações

Vantagens

- Menos operação directa de servidores.
- Execução gerida e integração com D1.
- Desenvolvimento local antes de publicar.

Limitações e cuidados

- Quotas, limites e dependência do fornecedor.
- Custos variam com utilização e plano.
- Segurança, dados e testes continuam a cargo da equipa.

Workers: pedidos e CPU. D1: leituras, escritas e armazenamento.

Referências: [4] preços Workers; [5] preços D1. Não foi realizada análise comparativa de custos.

Estado actual e caminho para publicação

CONCLUÍDO LOCALMENTE

- Frontend, API e persistência D1.
- Testes, build e pacote de publicação.
- Worker com Static Assets no mesmo servidor local.

CONFIGURAÇÃO EXTERNA

- Criar D1 remoto e configurar o ID real.
- Configurar domínio e Cloudflare Access.
- Aplicar migrações e publicar com Wrangler.

Local: npm.cmd run dev → http://127.0.0.1:5173

Fonte: docs/DEPLOYMENT.md. Nenhuma publicação remota foi realizada.

Conclusão e trabalhos futuros

O CloudExpense materializa os conceitos de serverless numa aplicação funcional e verificável localmente.

Contribuição

Interface, função e persistência integradas, com regras de negócio, segurança e evidências de teste.

Próximos passos

Publicar e medir o ambiente remoto.

Avaliar acessibilidade e carga.

Estudar contas e isolamento de dados.

Referências e fontes das evidências

[1] Cloudflare — How Workers works

developers.cloudflare.com/workers/reference/how-workers-works/

[2] Cloudflare — D1 overview

https://developers.cloudflare.com/d1/

[3] Cloudflare One — Validate JWTs

developers.cloudflare.com · Cloudflare One · Access · Validate JWTs

[4] Cloudflare — Workers pricing

https://developers.cloudflare.com/workers/platform/pricing/

[5] Cloudflare — D1 pricing

https://developers.cloudflare.com/d1/platform/pricing/

[6] CloudExpense — documentação e testes

docs/ARCHITECTURE.md · SERVERLESS.md · TESTING.md

Consulta: 11/10/2026. Capturas e diagramas: elaboração própria a partir do projecto.

Questões e discussão

A infraestrutura é gerida.
A responsabilidade pelo software continua.

Obrigado.

Ayrton Nhaguaga · Abelardo Miambo · Edmilson Conge

Lecc31 | Docente: Sergio Sambo